Revisiting Forest Proximities via Sparse Leaf-Incidence Kernels

Adrien Aumon* Guy Wolf* Kevin R. Moon† Jake S. Rhodes‡

Abstract. Decision forests induce supervised similarities through the partition structure of their trees. Yet forest proximity computation is still often treated as a quadratic operation in the number of samples, which limits scalability and restricts broader use in kernel and representation-learning pipelines. We introduce a unified view of leaf-collision forest proximities through a class of Separable Weighted Leaf-Collision (SWLC) kernels, showing that most existing proximities differ only in their weighting scheme while sharing a common sparse leaf-incidence structure. This yields an explicit leaf-space representation that clarifies their kernel interpretation and leads to an exact finite-sample sparse factorization of the proximity matrix, avoiding an explicit all-pairs comparison and reducing computation to sparse linear algebra over leaf collisions. We implement this framework in a memory-efficient Python library and show, both theoretically and empirically, that exact kernel computation scales near-linearly in time and memory under standard forest regimes. Benchmarks verify the predicted scaling behavior in practice across datasets, proximity definitions, and forest settings, and show that the resulting sparse leaf-space representation can also be used directly for fast task-aware embedding.

1 Introduction. Beyond their widespread use for classification and regression, Random Forests (RFs; [5, 6]) and other decision-forest methods such as Extremely Randomized Trees (ETs; [16]) and Gradient Boosted Trees (GBTs; [13]) also induce pairwise similarity measures through the partitioning structure of their trees. This notion of *forest proximity* was originally introduced by Breiman for RFs as the proportion of trees in which two observations fall into the same leaf [6]. Because splits are optimized for a supervised objective, these proximities yield a supervised similarity measure that implicitly reflects variable importance relative to auxiliary information.

Forest proximities have since attracted growing interest as a flexible supervised similarity for both data representation and model understanding across a wide range of tasks. They have been used for outlier detection, imputation, and general model exploration [38], and to induce task-aware geometries for supervised dimensionality reduction (DR), visualization, and alignment, including guided biological data exploration through 2D embeddings of multiple sclerosis subtype trajectories [37], RF-based autoencoding for out-of-sample embedding and reconstruction [1, 51], supervised manifold alignment [39], and RF biplots for variable interpretation in high-dimensional omics data [4]. They have also been used directly for prediction and decision making in prototype identification [46], tabular classification through RF-induced graph neural networks [12], time-series classification via proximity forests [43], patient-specific prediction in ICU data through local modeling [25], supervised similarity learning in finance [23], and matching treated and control subjects in causal inference [58]. Because proximities expose both instance-level relationships and feature-level structure, they are also useful for interpretability and explanation, complementing attribution methods such as SHAP [29] and LIME [40], with applications including visual feature contribution analysis [53], prediction-faithful explanations of RF and GBT models [38, 15, 41], and the characterization of copy-number-variation structure among pea aphid host races in evolutionary genomics [10].

Forest proximities are often used as implicit supervised kernel matrices in spectral and kernel-based methods. While a dot-product or feature-map view has been noted in some cases [51, 53, 34], there is still no general framework that explains when forest proximities admit explicit leaf-space representations, or how to use this structure in a systematic way for both methodology and scalability. Moreover, forest proximities remain hard to use at scale because many implementations require quadratic time or memory. Rhodes et al. [37] suggested that, for a fixed training point, one could restrict the computation to samples that share a leaf with it across trees, rather than compare it to all training samples, but they left this idea for future work.

We develop a general framework for forest proxim-

*Département de mathématiques et de statistique, Université de Montréal | Mila – Quebec AI Institute, Montreal, Quebec, Canada.

†Department of Mathematics and Statistics, Utah State University, Logan, Utah, USA.

‡Department of Statistics, Brigham Young University, Provo, Utah, USA (rhodes@stat.byu.edu).

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

ities. We introduce a family of leaf-based similarities called *Separable Weighted Leaf-Collision* (SWLC) proximities, and show that it includes most existing collision-based forest proximities. We also show that each member of this family has an explicit leaf-space representation, leading to an exact finite-sample sparse factorization of the proximity matrix in which computation is restricted to samples that collide in leaves. Under standard forest regimes, this yields near-linear time and space complexity in the number of samples. We support these results with a memory-efficient Python implementation and a broad empirical study across datasets, forest configurations, and kernel variants. The experiments confirm the expected runtime and memory trends in practice, and show that exact proximity computation remains feasible on datasets with hundreds of thousands of samples on standard CPU hardware. We also show that the same factorization enables the direct use of sparse leaf representations in downstream spectral methods, extending the value of forest proximities beyond scalable matrix construction alone.

2 Background.

2.1 Overview of leaf-collision proximities. Leaf-collision proximities are a family of similarity measures built from sample co-occurrence in terminal nodes across a tree ensemble. The guiding principle is simple: two points should be similar if they repeatedly fall into the same leaves across trees. At each node, the learning algorithm selects a split according to an optimization criterion such as entropy, Gini impurity, or mean squared error. Thus the routing path of a sample reflects a sequence of decisions based on features that are informative for the prediction task. This means that two similar samples followed the same routing path through a sequence of feature-based splits in multiple trees, and, therefore, remained indistinguishable under the task-adaptive partition learned by the ensemble, despite potential differences in uninformative features. Leaf collisions thus capture a notion of proximity that emphasizes task-relevant structure rather than label-free geometry.

The original RF proximity introduced by Breiman [5, 6] measures similarity as the fraction of trees in which two samples fall into the same terminal leaf. This simple leaf-collision principle remains widely used and competitive in practice [12]. Kernel variants such as KeRF [42] refine this definition by downweighting collisions in large leaves, yielding symmetric kernels with favorable properties for downstream methods such as diffusion [51]. Other extensions incorporate training-specific information. In particular, OOB proximities [27, 19] restrict collisions to trees where both samples are out-of-bag (OOB), reducing biases induced by bootstrap sampling and improving reliability in practice.

Building on these ideas, RF-GAP [38] combines OOB querying with leaf-mass normalization and in-bag multiplicities to better align proximities with the predictive geometry of the forest, achieving strong empirical performance across tasks while introducing asymmetry that can be symmetrized if needed [38, 37, 1]. Other variants instead reweight collisions using external criteria, such as instance hardness in RFProxIH [7]. Extensions to boosted trees have also been proposed, either through empirical per-tree weighting [46] or by deriving similarities directly from the additive regression model (AXIL; [15]). Refer to Appendix A for a more comprehensive literature review on forest proximities.

Each proposed proximity variant is motivated by a distinct goal, such as mitigating overfitting or faithful model explanations. However, these definitions have never been established on a common formal foundation: they are typically presented as stand-alone algorithmic recipes compared through downstream performance. In particular, it is often left implicit that collision-based proximities induce *kernels* that can be written as inner products in a high-dimensional leaf space. Moreover, different proximity measures largely amount to different choices of leaf-incidence feature maps (and associated weights) prior to taking a dot product. This perspective not only unifies their definitions but also has important implications for how these proximities can be computed and scaled. In particular, aside from recent work on GPU acceleration or heuristic block-sparse approximations [24], there has been limited effort to analyze the faithful computational cost of these inner-product constructions. As a result, collision-based proximities are often treated as inherently quadratic, $\mathcal{O}(N^2)$, in the number of samples, an assumption that has long discouraged their use at scale. This paper challenges that view by introducing a unifying algebraic framework that makes the underlying feature maps explicit, and by providing the corresponding complexity analysis needed to scale these measures to millions of samples.

2.2 General assumptions and notations. Let $\mathcal{D} = (X, Y) = \{(x_i, y_i)\}_{i=1}^{N} \subset \mathcal{X} \times \mathcal{Y}$ be a finite labeled training set. Rather than working on a specific forest training algorithm, we suppose the existence of an *ensemble context* $(\mathcal{T}, \theta)$ generated from $\mathcal{D}$. The first component, $\mathcal{T} = \{\mathcal{T}_t\}_{t=1}^{T}$, denotes the *ensemble topology*—the collection of decision tree structures that determine how samples are routed to leaves. The second component, θ, is a fixed *context object* providing auxiliary information related to the trained ensemble's

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

geometry, such as training hyperparameters, tree-wise descriptors, leaf-level statistics, or algorithm-dependent quantities derived during or after training.

Leaf indexing. For each tree $t \in \{1, \dots, T\}$, let L_t denote its number of leaves, and let $L := \sum_{t=1}^{T} L_t$ be the total number of leaves across the ensemble. The leaf index sets $\{\mathcal{L}_t\}_{t=1}^{T}$ form a partition of the global index set $\mathcal{L} = \{1, \dots, L\}$, i.e., $\bigsqcup_{t=1}^{T} \mathcal{L}_t = \mathcal{L}$, with $|\mathcal{L}_t| = L_t$. The ensemble topology $\mathcal{T}$ then induces, for each tree t, a leaf index map $\ell_t : \mathcal{X} \to \mathcal{L}_t$, where $\ell_t(x)$ is the unique global index of the leaf reached by x in tree t. Evaluating $\ell_t(x)$ requires traversing a single root-to-leaf path in tree t, and therefore costs $\mathcal{O}(h_t)$, where h_t denotes the depth of tree t.

Weight assignment $\boldsymbol{q}$. We define a weighting map

$$\boldsymbol{q} : \mathcal{X} \to \mathbb{R}^T, \qquad \boldsymbol{q}(x) = (q_1(x), \dots, q_T(x)),$$

where $q_t(x)$ is a scalar weight associated with sample x in tree t. In many proximity definitions, weights are determined by the fixed context and the leaf reached by x (more precisely, one could write $q_t(x \mid \theta, \ell_t(x))$). For readability, and to allow for arbitrary per-sample, per-tree weighting schemes, we simply write $q_t(x)$. Intuitively, $\boldsymbol{q}(x)$ encodes the contribution (or importance) of x across the ensemble. In typical constructions, evaluating $q_t(x)$ reduces to retrieving leaf-level statistics from θ in constant time. Our definition is deliberately more general, and allows q_t to involve nontrivial computations when needed. For example, in RFProxIH [7], q_t depends on an instance-hardness score that may require preprocessing or local neighborhood queries. Thus, defining $\boldsymbol{q}$ directly as a map on $\mathcal{X}$ keeps the weights locally tied to the ensemble partition while supporting flexible, sample-dependent assignments.

3 Methods.

We now introduce a unifying framework for forest proximities. By explicitly decoupling the topological collision from the contextual importance of each tree, we define a broad class of kernels that encompasses the majority of existing forest-based similarity measures.

3.1 SWLC proximities.

Definition 3.1 (Separable Weighted Leaf-Collision (SWLC) Proximity). *Given an ensemble context $(\mathcal{T}, \theta)$ and two weight assignments $\boldsymbol{q}$ and $\boldsymbol{w}$, a* separable weighted leaf-collision *(SWLC) proximity is a function $P_{\boldsymbol{q},\boldsymbol{w}} : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ of the form:*

$$P_{\boldsymbol{q},\boldsymbol{w}}(x, x') = \sum_{t=1}^{T} q_t(x)\, w_t(x')\, \mathbb{I}(\ell_t(x) = \ell_t(x')). \tag{3.1}$$

The term *separable* highlights that the weights assigned to the first argument x and the second argument x' are determined independently sample-wise within each tree. In the special case where $\boldsymbol{q} = \boldsymbol{w}$, the proximity becomes symmetric and, as shown later, admits a Gram representation in a sparse leaf-induced feature space.

Proposition 3.2 (Generality of SWLC proximities). *All leaf-collision based proximities introduced in Section 2.1 are symmetric or asymmetric SWLC proximities, except OOB and AXIL.*

Proof. The proof is obtained by identifying the specific ensemble context and weight assignments $\boldsymbol{q}$ and $\boldsymbol{w}$ required to recover each measure. For detailed derivations of these weights from their canonical definitions, we refer the reader to Appendix B. ∎

3.2 Sparse forest kernels.

While the summation in Eq. (3.1) provides a conceptual definition shared across most proximity definitions in the existing literature, its naive evaluation for all pairs (x_i, x_j) is computationally prohibitive. To reveal the underlying geometry and enable scalable computation, we translate the SWLC proximity into a high-dimensional dot product. As discussed in Section 2.1, while the literature contains various sophisticated proximities, their formulations are often presented as algorithmic procedures where the underlying inner-product structure remains implicit. We make this structure explicit by representing each sample as a sparse vector in the global leaf space.

Definition 3.3 (Weighted leaf-incidence map). *Given a weight assignment $\boldsymbol{q}$, the weighted leaf-incidence map $\boldsymbol{\phi}_{\boldsymbol{q}} : \mathcal{X} \to \mathbb{R}^L$ is defined by*

$$\boldsymbol{\phi}_{\boldsymbol{q}}(x) = \sum_{t=1}^{T} q_t(x)\, \mathbf{e}_{\ell_t(x)},$$

where $\mathbf{e}_j \in \mathbb{R}^L$ denotes the j-th canonical basis vector.

Lemma 3.4 (T-Sparsity). *Let $\|\cdot\|_0$ denote the number of nonzero elements (L_0 "norm"). For any $x \in \mathcal{X}$, the representation $\boldsymbol{\phi}_{\boldsymbol{q}}(x)$ satisfies*

$$\|\boldsymbol{\phi}_{\boldsymbol{q}}(x)\|_0 = \|\boldsymbol{q}(x)\|_0 \le T. \tag{3.2}$$

Proof. By Definition 3.3, we have

$$\begin{aligned} \|\boldsymbol{\phi}_{\boldsymbol{q}}(x)\|_0 = \left\| \sum_{t=1}^{T} q_t(x)\, \mathbf{e}_{\ell_t(x)} \right\|_0 &= \sum_{t=1}^{T} \|q_t(x)\, \mathbf{e}_{\ell_t(x)}\|_0 \\ &= \sum_{t=1}^{T} \mathbb{I}(q_t(x) \neq 0) = \|\boldsymbol{q}(x)\|_0 \le T, \end{aligned}$$

where the second equality follows from the fact that the supports of $\mathbf{e}_{\ell_t(x)}$ are pairwise disjoint across t, since $\ell_t(x) \neq \ell_{t'}(x)$ for any $t \neq t'$. ∎

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

LEMMA 3.5 (Constructive bilinear representation). *Let $P_{\boldsymbol{q},\boldsymbol{w}}$ be a SWLC proximity (Definition 3.1). Then, for all $x, x' \in \mathcal{X}$:*

$$P_{\boldsymbol{q},\boldsymbol{w}}(x, x') = \langle \boldsymbol{\phi}_{\boldsymbol{q}}(x), \boldsymbol{\phi}_{\boldsymbol{w}}(x') \rangle = \boldsymbol{\phi}_{\boldsymbol{q}}(x)^\top \boldsymbol{\phi}_{\boldsymbol{w}}(x').$$

Proof. See Appendix C. ∎

The bilinear form in Lemma 3.5 formalizes the distinct functional roles of the two arguments. In asymmetric SWLC proximities such as RF-GAP [38], the weight assignments $\boldsymbol{q}$ and $\boldsymbol{w}$ correspond to the roles of *query* and *reference*, respectively. The query weight $\boldsymbol{q}$ typically captures tree-level importance or sample-specific relevance, while the reference weight $\boldsymbol{w}$ captures leaf-level influence (e.g., density normalization). For out-of-sample (OOS) extension, an unseen sample x is by convention treated as the query, and its proximity to the training set is computed by projecting it via $\boldsymbol{\phi}_{\boldsymbol{q}}(x)$ and measuring its similarity against the gallery of reference representations $\{\boldsymbol{\phi}_{\boldsymbol{w}}(x_i)\}_{i=1}^N$ via the dot product in the leaf space.

While this latent dot-product structure has been observed in prior work, existing results are limited to a few specific tree-ensemble proximity definitions [51, 53]. Here, we generalize this view to the full SWLC family, including both symmetric and asymmetric constructions, where the similarity takes the form of a bilinear product that, in general, involves two distinct leaf-incidence maps, $\boldsymbol{\phi}_{\boldsymbol{q}}$ and $\boldsymbol{\phi}_{\boldsymbol{w}}$. The previously studied symmetric cases therefore correspond to a strict subset of SWLC, whereas our framework also covers more general weighted and directed proximity definitions. The proposition below is our central result, as it converts this bilinear form into a practical finite-sample sparse matrix factorization.

PROPOSITION 3.6 (Exact finite-sample sparse factorization). *Let $P_{\boldsymbol{q},\boldsymbol{w}}$ be a SWLC proximity (Definition 3.1). Define matrices $Q, W \in \mathbb{R}^{L \times N}$ by stacking the weighted leaf-incidence vectors,*

$$Q_{:i} := \boldsymbol{\phi}_{\boldsymbol{q}}(x_i) \in \mathbb{R}^L, \qquad W_{:j} := \boldsymbol{\phi}_{\boldsymbol{w}}(x_j) \in \mathbb{R}^L.$$

Then the training proximity matrix $P = [P_{\boldsymbol{q},\boldsymbol{w}}(x_i, x_j)] \in \mathbb{R}^{N \times N}$ admits the exact factorization

$$P = Q^\top W.$$

Moreover, each column i of Q (W) has exactly $\|\boldsymbol{q}(x_i)\|_0 \le T$ ($\|\boldsymbol{w}(x_j)\|_0 \le T$) nonzeros.

Proof. By Lemma 3.5, for all i, j,

$$P_{ij} = P_{\boldsymbol{q},\boldsymbol{w}}(x_i, x_j) = \boldsymbol{\phi}_{\boldsymbol{q}}(x_i)^\top \boldsymbol{\phi}_{\boldsymbol{w}}(x_j) = Q_{:i}^\top W_{:j}.$$

Thus $(Q^\top W)_{ij} = P_{ij}$. The column-sparsity statement follows from Lemma 3.4 applied to $\boldsymbol{\phi}_{\boldsymbol{q}}$ and $\boldsymbol{\phi}_{\boldsymbol{w}}$. ∎

COROLLARY 3.7 (Gram kernel under identical weight assignment). *Assume $\boldsymbol{q} = \boldsymbol{w}$ and let $Q \in \mathbb{R}^{L \times N}$ be defined by $Q_{:i} = \boldsymbol{\phi}_{\boldsymbol{q}}(x_i)$. Then the SWLC proximity matrix reduces to a Gram kernel:*

$$P = Q^\top Q = W^\top W,$$

and is therefore symmetric positive semidefinite.

Proof. If $\boldsymbol{q} = \boldsymbol{w}$, then both arguments of the proximity share the same feature map $\boldsymbol{\phi}_{\boldsymbol{q}}$. By Proposition 3.6, we have $Q = W$ by construction, and

$$P = Q^\top W = Q^\top Q = W^\top W.$$

Thus P is a Gram matrix induced by the embedding $x \mapsto \boldsymbol{\phi}_{\boldsymbol{q}}(x)$, which implies symmetry and positive semidefiniteness, since $c^\top P c = \|Qc\|_2^2 \ge 0$ for any $c \in \mathbb{R}^N$. ∎

Beyond enabling scalable proximity computation, this decomposition is also a structural result: it makes the feature maps underlying SWLC proximities explicit and shows that these proximities can be interpreted as inner products between leaf-based representations. This perspective clarifies why SWLC proximities integrate naturally with kernel methods such as kernel PCA and kernel support vector machines (SVM), rather than behaving as black-box similarities. It also explains why kernel PCA could be successfully applied to forest-based proximity matrices in prior work [37, 36].

In particular, in the symmetric case, the singular value decomposition (SVD) theorem shows that the spectral structure of the full proximity matrix P can be recovered directly from the explicit leaf-incidence matrix Q [22]. As a result, spectral methods formulated on P can be implemented equivalently on Q, without ever forming the dense matrix P. This is particularly attractive computationally, since it allows one to exploit sparse linear algebra directly in leaf space [26, 17, 50].

Kernel-style methods have also been generalized to asymmetric kernels [21, 45, 47], which can capture richer directional relationships. Proposition 3.6 thus provides a principled foundation for developing general supervised kernel methods based on directed forest proximities.

3.3 Complexity analysis. Computing P from scratch involves four stages: (i) training the ensemble $\mathcal{T}$, (ii) computing the metadata θ required by the weight assignments $(\boldsymbol{q}, \boldsymbol{w})$, (iii) constructing the sparse factors Q and W, and (iv) evaluating the product $Q^\top W$. For standard tree ensemble methods, if $\bar{h} := \frac{1}{T}\sum_{t=1}^T h_t$

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

denotes the average tree height, training and routing scale as $\mathcal{O}(NT\bar{h})$ under common implementations [28, 18]. Likewise, when the metadata is sample-wise, tree-wise, or leaf-wise, as in most proximity definitions, it can typically be obtained by routing followed by local leaf aggregation, with cost at most $\mathcal{O}(NT\bar{h}) + \mathcal{O}(NT)$. Thus, these preprocessing steps do not introduce a quadratic scaling in N.

The main remaining bottleneck is the proximity computation itself. A naive implementation would still require evaluating all N^2 sample pairs across trees, yielding $\mathcal{O}(N^2T)$ complexity. We now show that the sparse product $Q^\top W$ avoids this quadratic cost by exploiting the sparsity patterns of Q and W, so that computation is restricted to sample pairs that collide in at least one leaf.

Building Q and W. Although Q and W are mathematically $L \times N$, Proposition 3.6 implies only $\mathcal{O}(NT)$ memory is required in compressed sparse format. Constructing them requires routing each sample through each tree. Thus, this stage costs $\mathcal{O}(NT\bar{h})$.

Sparse product $Q^\top W$. The computational gain comes from the fact that sparse multiplication does not evaluate all N^2 sample pairs densely. For any pair (i, j), $P_{ij} = \boldsymbol{\phi_q}(x_i)^\top \boldsymbol{\phi_w}(x_j)$ is non-zero only if the two leaf-incidence vectors share at least one active coordinate, that is, only if x_i and x_j fall in the same leaf of at least one tree. This is precisely the mechanism exploited by `scipy`'s sparse routines [50]: the product is accumulated only through shared non-zero column indices, rather than by checking all N^2 pairs explicitly.

To quantify this, fix a sample x_i and a tree t. Instead of comparing x_i to all N samples, tree t only induces interactions with the samples that fall in the same leaf as x_i. If $n_{t,\ell_t(x_i)}$ denotes the number of samples in that leaf, then the work associated with sample x_i across all trees is proportional to $\sum_{t=1}^{T} n_{t,\ell_t(x_i)}$. Summing over all samples yields the global sparse multiplication cost $\mathcal{O}\Big(\sum_{i=1}^{N}\sum_{t=1}^{T} n_{t,\ell_t(x_i)}\Big) = \mathcal{O}(NT\bar{\lambda})$, where

$$\bar{\lambda} := \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T} n_{t,\ell_t(x_i)}$$

is the average number of same-leaf interactions per sample and per tree.

The combined routing and sparse product cost are:

$$\begin{aligned} \text{TIME} \quad &: \quad \mathcal{O}(NT\bar{h}) + \mathcal{O}(NT\bar{\lambda}) = \mathcal{O}\big(NT(\bar{h} + \bar{\lambda})\big), \\ \text{SPACE} \quad &: \quad \mathcal{O}(NT) + \mathcal{O}(NT\bar{\lambda}) = \mathcal{O}\big(NT(1 + \bar{\lambda})\big). \end{aligned}$$

Thus, the complexity of the factorized computation is governed by N, T, the average tree height $\bar{h}$, and the average leaf-collision factor $\bar{\lambda}$. In standard regimes where $T \ll N$, $\bar{h} \approx \log N$ for balanced trees, and $\bar{\lambda} \ll N$ for sufficiently deep trees, the full pipeline is near-linear in the sample size (or at least sub-quadratic).

Remark 3.8. These bounds can be tightened for specific sparse weight assignments, but we omit this refinement for simplicity, as our focus is the scaling behavior of the full SWLC family. See Section 4.2 for an empirical comparison of different weighting schemes.

Remark 3.9. The same reasoning applies to OOS extension. If $Q_{\text{new}} \in \mathbb{R}^{N_{\text{new}} \times L}$ denotes the sparse leaf-incidence matrix of N_{new} unseen samples, then computing cross-proximities via $Q_{\text{new}}^\top W$ costs $\mathcal{O}(N_{\text{new}}T\bar{h}) + \mathcal{O}(N_{\text{new}}T\bar{\lambda}_{\text{ext}})$, where the first term accounts for routing the new samples through the ensemble and building Q_{new}, and the second term accounts for the sparse multiplication. Here, $\bar{\lambda}_{\text{ext}}$ denotes the average number of reference samples sharing a leaf with a queried sample, per tree. Hence, OOS proximity extension is also near-linear in the number of queried samples when $\bar{h}$ and $\bar{\lambda}_{\text{ext}}$ are bounded or slowly growing.

4 Results.

We implemented our sparse forest kernel in a user-friendly Python interface built on top of standard `scikit-learn` [35] forest estimators, which made it easy to integrate with existing workflows while keeping the API familiar. All experiments were run on a CPU compute node. See Appendices D and E for details on the computing environment and implementation. In the experiments below, we consider 11 medium- to large-scale labeled datasets spanning diverse domains, feature dimensions, and numbers of classes, to illustrate the relevance of our sparse kernel decomposition across a broad range of settings. A summary of the datasets is provided in Appendix F.

4.1 Asymptotic separability of OOB proximity.

Proposition 3.2 shows that standard OOB proximities are non-separable. We propose an alternative, $\tilde{P}_{\text{oob}}$, which achieves separability by normalizing leaf collisions with the proxy $S(x)S(x')/T$ instead of the shared OOB tree count $S(x, x')$. This formulation enables scalable computation via sparse kernel decomposition and explicit leaf coordinates while remaining asymptotically equivalent to the standard definition as T and N increase (see Appendix G).

We empirically validate this asymptotic separability using a subset of SignMNIST [30] (letters A–K). We evaluate the average ratio $R(x, x') = \frac{S(x,x')}{S(x)S(x')/T}$ across distinct samples where $S(x, x') > 0$, varying training size N (5%–50%) and forest size T (60–150). Results are shown in Fig. 4.1, and support our Proposition G.1: the ratio stabilizes as T grows, and approaches 1 from below as N increases. In standard regimes, the approximation

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

error is negligible, providing a faithful and scalable alternative to standard OOB formulations (Fig. 4.2).

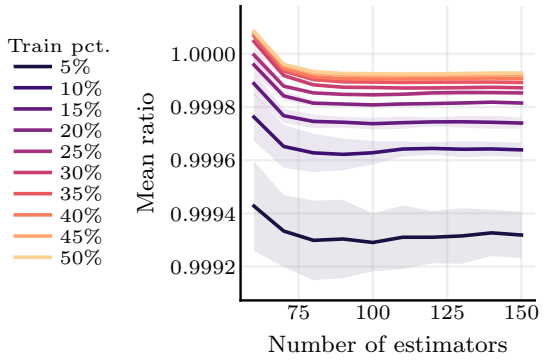


Figure 4.1: Mean ratio $R(x, x') = \frac{S(x,x')}{S(x)S(x')/T}$ with standard deviations on Sign MNIST (A–K) vs. number of trees T. Curves represent training sizes N from 5% to 50%. As T and N increase, the ratio converges toward 1 from below, validating the asymptotic separability of Proposition G.1 and the fidelity of the $\tilde{P}_{\text{oob}}$ proxy.

4.2 Empirical scaling of sparse forest kernel computation. We empirically validate the near-linear time and memory scaling in the number of samples established in Section 3.3, across a range of settings. We use standard RF training as implemented in `scikit-learn`'s `RandomForestClassifier`, together with RF-GAP weight assignments. For evaluation, each dataset is split into train and test sets using a 10% stratified split (or predefined splits when available).

We analyze the time and memory required to compute the full kernel as the number of training samples increases, under three axes of variation: (i) across nine datasets of varying scale, including the Higgs dataset [2] with up to 10 million samples (Fig. 4.2, top), (ii) across four proximity definitions—original, KeRF, our *separable* OOB, and RF-GAP (Fig. 4.2, middle); and (iii) across the minimum leaf size parameter $n_{\min}$ (Fig. 4.2, bottom). We performed (i) and (ii) on Covertype. Additional ablations, including experiments with constrained tree depth and additional datasets, are provided in Appendix H. We also report, in Appendix I, the test accuracy of proximity-weighted predictions as a sanity check to assess the usefulness of the mined kernels for downstream classification tasks.

Reported runtime and memory correspond to the cost of constructing and storing the cached metadata, query maps, and the resulting sparse kernel. Forest training is excluded, as it is independent of our decomposition and does not dominate computational cost.

Overall, both runtime and memory exhibit near-linear scaling (with slopes well below 2 and often close to 1), in agreement with our theoretical analysis. On the Higgs dataset, computing the full kernel on a 1M-sample subset requires under one minute and less than 16GB of memory, making the approach practical on standard hardware. Across proximity definitions, original and KeRF show similar scaling, as both rely on full leaf collisions. In contrast, the OOB kernel is the most scalable, since it only considers OOB–OOB interactions, while GAP lies in between due to its OOB–in-bag structure. This aligns with Remark 3.8, which suggests that tighter bounds can be obtained by accounting for sparsity induced by specific weighting schemes. Finally, increasing the minimum leaf size degrades performance compared to unconstrained forests, but near-linear scaling is preserved over a broad range of $n_{\min}$, making the approach robust to practical hyperparameter tuning.

4.3 Manifold learning on leaf coordinates. Beyond enabling scalable full kernel computation, our explicit kernel decomposition opens another promising direction: injecting the structure learned by forest models into downstream manifold learning methods that typically operate directly on the feature space.

Our leaf maps live in a space of dimension L, which in the worst case can be as large as NT. Although each vector is only T-sparse, most manifold learning pipelines expect dense inputs, and therefore do not naturally handle sparse leaf coordinates without an intermediate DR step. The most common choice is PCA, which is often used as a preprocessing step for nonlinear DR methods. Fortunately, the `scikit-learn` implementation of PCA provides an ARPACK solver [26] that can operate on sparse inputs through linear operators, without explicitly centering the full matrix in leaf space. Since the runtime of many manifold learning methods is dominated by neighbor search and graph construction, computing Leaf PCA coordinates is not typically the bottleneck. This makes leaf coordinates a practical way to incorporate forest-based supervision into existing pipelines.

Our decomposition also provides a useful conceptual point of view. When standard forest proximities are used as precomputed kernels, the resulting DR pipeline implicitly restricts to linear leaf-collision relationships. In contrast, our explicit leaf coordinates expose the same structure in a vector representation, making it possible to go beyond standard kernel-based workflows and apply more general nonlinear manifold learning methods directly in the leaf space. We explore this here.

We consider the FashionMNIST dataset [54], with 60,000 training samples and 10,000 test samples. We study two families of DR pipelines: PCA alone, PCA followed by UMAP [31], and PCA followed by PHATE [32], with 50 principal components used as preprocessing for UMAP and PHATE. The second fam-

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

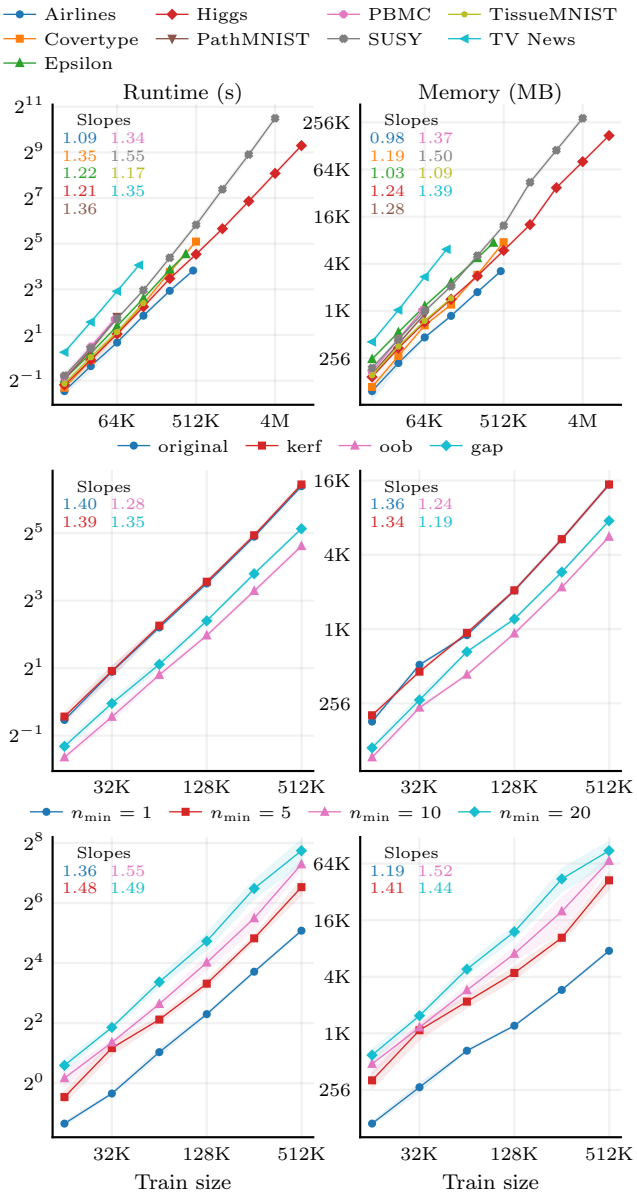


Figure 4.2: Log–log runtime and memory scaling of exact kernel computation with sample size, across datasets, proximity methods, and minimum leaf sizes. Shaded bands show standard deviations, and slopes are from fitted linear regressions. Overall, computation scales near-linearly with sample size, with slopes well below 2. RF-GAP and our separable OOB kernel scale more favorably, consistent with the extra sparsity induced by their weighting schemes. Larger minimum leaf sizes degrade scaling, but remain practical and robust across a reasonable range of values.

ily applies the same pipelines to sparse leaf coordinates instead of raw image features. For simplicity, we use KeRF leaf coordinates, which ensure symmetry for the PCA step. We otherwise use default parameters for all methods, except that we set $k = 30$ for UMAP and PHATE to ensure graph connectedness during k-NN construction. Each pipeline is trained on the raw training images, and both train and test samples are embedded in two dimensions. In Fig. 4.3, we report the runtime of each pipeline and the average test embedding k-NN accuracy for $k = 5, 10, 20$, using the training embedding as reference. We also display the corresponding classwise medoid images on each embedding.

All DR pipelines benefited from leaf coordinates in terms of class separability, especially PCA. This is expected: raw PCA is purely linear, whereas Leaf PCA introduces nonlinearity through the leaf-incidence maps, making it analogous to kernel PCA. While raw PCA produces a noisy embedding, Leaf PCA reveals two dominant directions that are both class-aware and semantically meaningful: a horizontal direction related to accessories, with bags and, farther away, shoes, and a vertical direction related to broader clothing types such as dresses and trousers. This makes Leaf PCA a strong tool for visualizing forest-guided global structure.

Leaf UMAP and Leaf PHATE act more as supervised refinements of their unsupervised counterparts. In raw UMAP and raw PHATE, classes such as coat, shirt, and pullover, or ankle boot, sandal, and sneaker, remain partly mixed. Their leaf-based versions separate these groups more clearly without distorting their relationships. Leaf UMAP emphasizes classwise cluster separation, while Leaf PHATE highlights connections between clusters and class-aware trajectories. Leaf PHATE also shows a progressive transition among coats, shirts, and pullovers in the top-right part of the embedding. This is meaningful, since these classes can be viewed as related variants of a broader clothing type. From a computational perspective, the runtime of Leaf UMAP and Leaf PHATE is dominated by the UMAP or PHATE stage, while the Leaf PCA step is negligible by comparison. We provide a similar visual comparison on SignMNIST (A–K) in Appendix J. Overall, the exposed leaf space provides a scalable coordinate system that integrates supervision smoothly into existing DR pipelines.

5 Discussion. Forest proximities have long been used as supervised similarities for visualization, prediction, matching, and explanation, but their broader use has been limited by their quadratic scalability. Our work addresses this scalability. On the conceptual side, we introduce the SWLC framework, which unifies a broad class of forest proximities through a common leaf-collision form, making their feature-space structure explicit. In symmetric cases, this yields a sparse Gram representation and a direct kernel view. In asymmetric

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

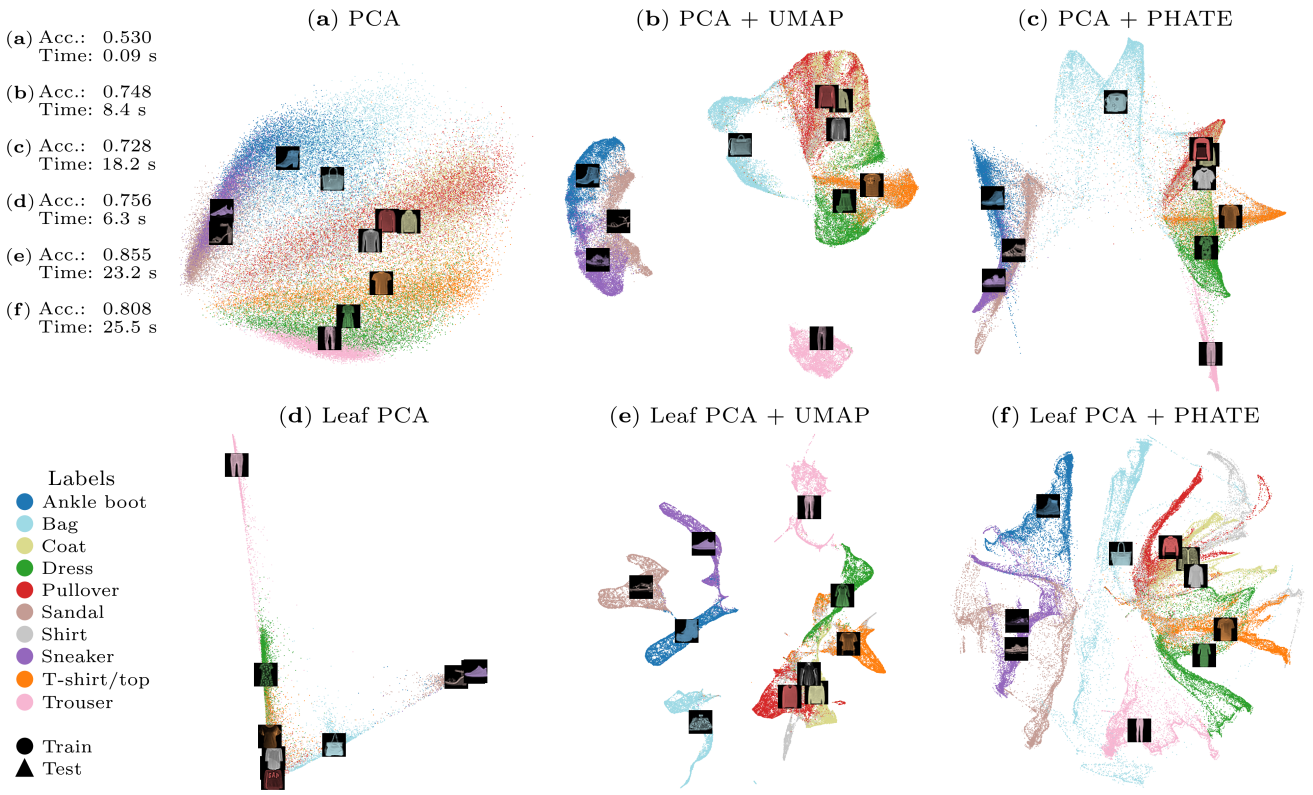


Figure 4.3: Two-dimensional train and test embeddings of Fashion MNIST samples obtained from raw flattened pixel features using (a) PCA, (b) PCA followed by UMAP, and (c) PCA followed by PHATE, compared with the same pipelines applied to sparse forest leaf coordinates in (d–f). The top-left of each panel reports runtime and test k-NN accuracy. Images show class-wise medoids in the embedding space. Overall, sparse leaf coordinates provide a scalable, denoised, task-aware representation for existing manifold learning pipelines.

cases, it yields a bilinear leaf-space view that also covers directed proximities. Computationally, this leads to an exact sparse factorization that avoids explicit pairwise comparison across all samples. Our experiments show that this gives strong gains in runtime and memory, and that these gains remain stable across datasets, ensemble types, kernel variants, and forest depth settings.

Because the factorization exposes explicit sparse leaf-incidence representations, spectral methods can operate directly on the leaf maps instead of on the full proximity matrix. In the symmetric case, this gives the same uncentered spectral structure as the kernel route, while fitting naturally within standard sparse linear algebra pipelines. Our results confirm that this direct leaf route preserves the relevant geometry and reduces computational cost.

A limitation is that our full kernel complexity bounds rely on computation being restricted to leaf-colliding samples. This is favorable in the usual regime of reasonably deep trees, but may weaken when trees are too shallow, and leaves become large. Still, our empirical results suggest that the near-linear behavior is robust in a broad practical regime.

The framework opens several natural directions. One is to enrich existing separable proximities, e.g., by adding leaf-quality statistics such as impurity measures, or by combining ideas from different ensemble paradigms such as boosting and bagging [14]. Another is to move from fixed weighting rules to learned ones while keeping the forest topology fixed. This would yield sample-dependent forest kernels that remain faithful to the supervised geometry induced by the ensemble, while preserving sparsity. In this sense, our framework also points toward forest-based kernel learning, including tree reweighting schemes related to multiple kernel learning [44] and differentiable optimization on top of fixed tree structures [20]. Finally, and perhaps the most promising direction, is to pursue our preliminary results in Section 4.3 on the integration of the leaf coordinates into downstream manifold learning.

On the implementation side, we plan to broaden the range of supported ensemble contexts and weighting

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

schemes, to support user-defined forest kernels, and to accelerate the full pipeline further through GPU support for training, metadata extraction, and sparse proximity computation. We expect this work to move forest proximities from a set of specialized algorithms to a general, scalable, and explicit kernel framework.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

A Extended literature review on forest proximities.

A.1 Proximities based on leaf collisions. The original forest proximity, introduced by Breiman [5, 6], defines similarity between two samples as the proportion of trees in which they fall into the same terminal leaf. It is the most direct and canonical notion of leaf-collision similarity: two points are considered close when the ensemble repeatedly groups them together through learning a feature space partition. Despite its simplicity, the original proximity remains widely used and can be highly competitive in practice in specific contexts [12].

A normalized variant from the kernel random forest (KeRF) literature [42] refines the original definition by downweighting collisions that occur in large leaves. The intuition is that sharing a very large terminal cell provides a weaker notion of task-aware similarity than sharing a small, exclusive leaf. This leaf-mass normalization preserves symmetry and yields a kernel with useful stochastic properties, making it particularly attractive for downstream diffusion-based methods [51].

Several variants modify collision counting using additional information from the training procedure, such as bootstrap aggregation. The original RF proximities can be biased because they ignore each observation's bootstrap status, that is, whether it was in-bag or OOB for a given tree. Since trees are typically grown to purity, in-bag samples from different classes almost always end up in different terminal leaves, which can artificially amplify class separation in the resulting proximity matrix. OOB proximity [27, 19] addresses this effect by counting leaf collisions only in trees for which both samples are simultaneously OOB. This definition is used in the `randomForest` package [27] for the R programming language [48].

Along this line, RF-GAP [38] combines OOB querying with leaf-mass normalization and replaces binary indicators with the full per-tree in-bag multiplicity of each training sample. The resulting affinities are more closely aligned with the geometry that the forest learns for prediction. In particular, RF-GAP induces a proximity-weighted predictor that exactly recovers the RF OOB predictions [38]. It has been shown to outperform OOB proximities and several classical alternatives in tasks such as classification, regression, imputation, and outlier detection [38]. A drawback is that RF-GAP is generally asymmetric, making it incompatible with standard downstream methods requiring symmetry, although a symmetric variant can be obtained by averaging the two directions while retaining much of the practical benefit of the directed formulation [38, 37, 1].

Other RF-based variants also retain the same leaf-collision principle but reweight collisions using additional sample-level information. RFProxIH is an adaptation of the RFDisIH dissimilarity [7] for multi-view classification. Rather than recovering the forest's own geometry, it uses the forest partition as a scaffold and re-weights leaf collisions using an external notion of instance hardness (IH). It is therefore best viewed as a task-specific reweighting of leaf collisions, rather than as a forest-native interpretability tool. The authors reported strong classification performance for RFDisIH relative to other multi-view baselines, but did not compare it to other random-forest proximity definitions in common proximity-driven applications such as visualization or imputation.

The literature on proximities for GBTs is more limited, but has gained traction recently. Tan et al. [46] extended the original leaf-collision proximity to boosted ensembles by weighting each tree's collision indicator by that tree's contribution to the additive model. They showed that this tree-weighted proximity can outperform Euclidean and RF-induced distances for prototype selection with k-medoids. More recently, the Additive eXplanations with Instance Loadings (AXIL [15]) framework derives a supervised proximity for GBTs by rewriting the model prediction as an additive combination of training targets and interpreting the resulting weights as a similarity measure. Unlike purely heuristic tree-weighting schemes, AXIL is closer in spirit to RF-GAP: it incorporates leaf-size normalization so that the induced weights better explain the numerical output of the fitted regression model, and it discusses extensions to classification as future work. The authors further show that these AXIL weights provide useful, complementary instance-based explanations in a smoking-prevalence application, alongside feature-based tools such as SHAP [29] and LIME [40].

A.2 Other forest proximities. Proximity-based kernel (PBK; [11]) departs from strict leaf collisions by incorporating the tree topology: instead of assigning similarity only when two samples land in the same leaf, it measures how far apart they are within a tree. PBK defines the proximity between two observations as the average, across trees, of an exponentially decaying function of the number of branches separating them. This yields a nonzero similarity even between points in different leaves, which can capture finer graded relationships than the original proximity. The authors report improved robustness over the original proximity for shallow forests when used with SVM classifiers. The tradeoff is that PBK introduces a decay hyperparameter and is substantially more expensive to compute, since it requires evaluating branch distances for many pairs; in practice, its computational cost scales poorly, and reported gains appear mostly confined to small ensembles.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

Random Partitioning Kernel (RPK [9]) views tree-based kernels through the lens of random data partitions. Rather than relying primarily on terminal node co-occurrence, RPK samples a random tree height and partitions the data using that intermediate level of the tree, effectively defining similarity through co-membership in a randomly chosen coarse partition. The authors compare this RF kernel to standard kernels (e.g., linear, RBF) using a log-likelihood based evaluation, reporting favorable performance in many cases. They illustrate the induced structure with 2D visualizations through principal component analysis (PCA); however, they do not position the method as a proximity definition alongside existing forest proximities, and an implementation is not publicly available. Both PBK and RPK are therefore less commonly used in the proximity literature and, importantly, they are not strictly leaf collision-based. While they are interesting in their own right, we do not align our framework with them here, and leave extensions in this direction for future work.

B Forest proximities as SWLC kernels. In this section, we show that several common forest proximity measures can be expressed as instances of the Separable Weighted Leaf-Collision (SWLC) family defined in Definition 3.1. For each method, we specify an ensemble context $(\mathcal{T}, \theta)$ and a corresponding weight assignment $(\boldsymbol{q}, \boldsymbol{w})$ that recovers the target proximity. While we fix the ensemble context for each method, the associated weight assignment is generally not unique: the same proximity can often be obtained from multiple valid choices of $(\boldsymbol{q}, \boldsymbol{w})$. When the resulting SWLC proximity is symmetric, we present identical weight assignments for the two arguments, which is a sufficient condition for symmetry.

B.1 Original proximities. Let $\mathcal{T}$ be a trained decision forest with T trees. The original proximity (introduced in the context of RFs) depends only on the ensemble topology ($\theta = \emptyset$). It is defined by

$$P_{\text{original}}(x, x') = \frac{1}{T} \sum_{t=1}^{T} \mathbb{I}\big(\ell_t(x) = \ell_t(x')\big).$$

This is an instance of the SWLC form with weights

$$q_t(x) = w_t(x) = \frac{1}{\sqrt{T}}, \qquad t = 1, \ldots, T.$$

In particular, P_{original} is a symmetric SWLC proximity since $q = w$.

B.2 KeRF proximities. KeRF [42] augments the ensemble with leaf-level statistics. $\mathcal{T}$ is a trained decision forest and $\theta = M \in \mathbb{R}^L$, where $M(j)$ denotes the mass (number of training samples) in leaf j. The proximity is defined as

$$P_{\text{KeRF}}(x, x') = \frac{1}{T} \sum_{t=1}^{T} \frac{\mathbb{I}\big(\ell_t(x) = \ell_t(x')\big)}{M\big(\ell_t(x)\big)}.$$

Setting

$$q_t(x) = w_t(x) = \frac{1}{\sqrt{T\, M(\ell_t(x))}},$$

we recover P_{KeRF}. Indeed, whenever $\ell_t(x) = \ell_t(x')$, the denominator satisfies $M(\ell_t(x)) = M(\ell_t(x'))$, so the product $q_t(x)\, w_t(x')$ yields $1/(T\, M(\ell_t(x)))$ on colliding leaves and 0 otherwise. Hence, KeRF is a symmetric SWLC proximity.

B.3 OOB proximities. OOB proximities incorporate bootstrap information. $\mathcal{T}$ is a trained decision forest via bootstrap aggregation, and θ includes (i) per-sample, per-tree OOB indicators $o_t(\cdot) \in \{0, 1\}$, and (ii) the *shared* OOB tree counts

$$S(x, x') = \sum_{t=1}^{T} o_t(x)\, o_t(x'),$$

which record the number of trees for which *both* samples are OOB. The OOB proximity is defined as

$$P_{\text{oob}}(x, x') = \frac{1}{S(x, x')} \sum_{t=1}^{T} o_t(x)\, o_t(x')\, \mathbb{I}\big(\ell_t(x) = \ell_t(x')\big).$$

The normalization $S(x, x')$ depends jointly on the pair (x, x') and, in general, does not admit a separable factorization into a function of x times a function of x' alone. As a result, P_{oob} is not, in general, an SWLC proximity.

Nevertheless, as shown in Proposition G.1, it admits an asymptotically symmetric separable formulation $q_t(x) = w_t(x) = \frac{\sqrt{T}}{S(x)}$ where $S(x) = \sum_t o_t(x)$ is the per-sample OOB tree counts.

B.4 RF-GAP proximities. RF-GAP [38] incorporates both leaf-mass normalization and sample-dependent reweighting based on bootstrap status. It is designed so that, when used as weights in a proximity-weighted predictor, the resulting estimator matches the forest's OOB predictions exactly. $\mathcal{T}$ is a trained decision forest via bootstrap aggregation, and θ contains (i) leaf in-bag masses $M_{\text{in-bag}} \in \mathbb{R}^L$, (ii) per-sample, per-tree OOB indicators $o_t(\cdot)$ and in-bag multiplicities $c_t(\cdot)$, and (iii) per-sample OOB tree counts $S(\cdot)$. The RF-GAP proximity is

$$P_{\text{gap}}(x, x') = \frac{1}{S(x)} \sum_{t=1}^{T} \frac{o_t(x) c_t(x')\, \mathbb{I}\big(\ell_t(x) = \ell_t(x')\big)}{M_{\text{in-bag}}(\ell_t(x'))}.$$

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

This admits an SWLC representation by taking

$$q_t(x) = \frac{o_t(x)}{S(x)}, \qquad w_t(x) = \frac{c_t(x)}{M_{\text{in-bag}}(\ell_t(x))}.$$

Therefore, P_{gap} is a SWLC proximity, and is generally asymmetric.

B.5 IH proximities. RFProxIH incorporates IH through the k-disagreeing neighbors (DN) score. $\mathcal{T}$ is a trained decision forest, and θ contains the collection of tree-dependent hardness functions $kDN_t(\cdot)$ defined relative to the labeled training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$. The proximity is

$$P_{\text{ih}}(x, x') = \frac{1}{T}\sum_{t=1}^{T}\big(1 - kDN_t(x')\big)\,\mathbb{I}\big(\ell_t(x) = \ell_t(x')\big).$$

Here,

$$kDN_t(x) = \frac{1}{k}\,|\{\, j : x_j \in kNN_t(x) \subset X,\ y_j \neq y \,\}|\,,$$

where $kNN_t(x)$ denotes the set of k nearest neighbors of x computed in the tree-dependent feature subspace induced by the splitting variables along the decision path leading to the relevant leaf in tree t. This can be written in SWLC form by choosing

$$q_t(x) = \frac{1}{T}, \qquad w_t(x) = 1 - kDN_t(x).$$

Therefore, P_{ih} is an SWLC proximity and is generally asymmetric. A symmetric variant can be obtained, for example, by distributing the hardness factor across both arguments (e.g., using a symmetric combination of $kDN_t(x)$ and $kDN_t(x')$).

B.6 Boosted proximities. In GBTs, each tree t is associated with a nonnegative weight w_t reflecting its contribution to the additive model. $\mathcal{T}$ is a trained sequential forest via gradient boosting, and $\theta = \{w_t\}_{t=1}^T$. The corresponding proximity [46] is

$$P_{\text{boosted}}(x, x') = \frac{\sum_{t=1}^{T} w_t\,\mathbb{I}\big(\ell_t(x) = \ell_t(x')\big)}{\sum_{s=1}^{T} w_s}.$$

This admits an SWLC representation by setting

$$q_t(x) = w_t(x) = \sqrt{\frac{w_t}{\sum_{s=1}^{T} w_s}}.$$

Therefore, P_{boosted} is a symmetric SWLC proximity, since it uses identical weight assignments ($q = w$).

B.7 AXIL proximities Additive eXplanations with Instance Loadings (AXIL [15]) follows a similar spirit to RF-GAP, but in the setting of gradient-boosted regression trees. Rather than relying on a heuristic proximity, AXIL computes exact training-instance weights by recursively tracing the residual-fitting dynamics of the boosted ensemble.

Let $\mathcal{T}$ be the trained boosted forest, and let θ include the leaf-incidence structure $\{\ell_t(\cdot)\}_{t=1}^T$, the leaf-mass attribution $M \in \mathbb{R}^L$, and the recursively defined weights $\{P_t(m, x)\}_{t,m,x}$. To match the left-hand-side query convention used for RF-GAP, we write the AXIL proximity with the query instance in the first argument. Thus, for a query instance x and a training instance x', we define

$$P_{\text{AXIL}}(x, x') = \frac{1}{N} + \sum_{t=1}^{T}\sum_{m=1}^{N} P_t(m, x')\,\frac{\mathbb{I}\big(\ell_t(x_m) = \ell_t(x)\big)}{M\big(\ell_t(x)\big)},$$

where $P_t(m, x')$ are defined recursively as

$$P_t(m, x') = \lambda\Bigg[\frac{\mathbb{I}\big(\ell_t(x_m) = \ell_t(x')\big)}{M\big(\ell_t(x')\big)} - \sum_{k=1}^{N}\frac{\mathbb{I}\big(\ell_t(x_m) = \ell_t(x_k)\big)}{M\big(\ell_t(x_k)\big)}\,G_{t-1}(k, x')\Bigg],$$

with

$$G_{t-1}(k, x') = \frac{1}{N} + \sum_{s=1}^{t-1} P_s(k, x'),$$

and where λ denotes the learning rate of the boosting procedure.

Although the outer term still involves leaf collisions with the query x, the coefficients $P_t(m, x')$ depend on the accumulated residual history through G_{t-1}. Since the contribution of tree t cannot be expressed through independent per-tree weights $q_t(x)$ and $w_t(x')$. AXIL is not a SWLC proximity, and is generally asymmetric.

C Justification of Lemma 3.5.

Proof. Fix $x, x' \in \mathcal{X}$. By Definition 3.3,

$$\begin{aligned}
\boldsymbol{\phi}_{\boldsymbol{q}}(x)^\top \boldsymbol{\phi}_{\boldsymbol{w}}(x') &= \left[\sum_{t=1}^{T} q_t(x)\,\mathbf{e}_{\ell_t(x)}\right]^\top \left[\sum_{s=1}^{T} w_s(x')\,\mathbf{e}_{\ell_s(x')}\right] \\
&= \sum_{t=1}^{T}\sum_{s=1}^{T} q_t(x)\,w_s(x')\,\mathbf{e}_{\ell_t(x)}^\top \mathbf{e}_{\ell_s(x')} \\
&= \sum_{t=1}^{T} q_t(x)\,w_t(x')\,\mathbf{e}_{\ell_t(x)}^\top \mathbf{e}_{\ell_t(x')} \\
&= \sum_{t=1}^{T} q_t(x)\,w_t(x')\,\mathbb{I}(\ell_t(x) = \ell_t(x')) \\
&= P_{\boldsymbol{q},\boldsymbol{w}}(x, x'),
\end{aligned}$$

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

where the third equality uses that $\mathbf{e}_{\ell_t(x)}^\top \mathbf{e}_{\ell_s(x')} = 0$ since $\ell_t(x) \neq \ell_s(x')$ for $t \neq s$, and the fourth equality uses $\mathbf{e}_a^\top \mathbf{e}_b = \mathbb{I}(a = b)$. □

D Implementation overview. We implemented a unified `ForestKernel` API in Python that organizes the computation into three main steps. First, the user instantiates the object by specifying a `model_type`, which selects the underlying tree ensemble backend, and a `kernel_method`, which selects the proximity construction. Internally, a common adapter layer exposes the ensemble-specific quantities needed by the kernel builders, such as leaf assignments, node counts, OOB masks, in-bag multiplicities, and tree weights when available.

Second, the object is fitted in two consecutive stages. The method `fit_forest` trains the underlying ensemble only, while `build_kernel_cache` constructs the kernel-specific cached quantities and the sparse reference-side map W. In practice, the public `fit` method simply combines these two stages. The fitted kernel is represented in factored form as $K = QW^\top$, where Q and W denote the query-side and reference-side maps[1]. Symmetric kernels correspond to the special case $Q = W$, while asymmetric constructions such as RF-GAP are handled through distinct query and reference maps.

Third, once the cache is built, the API exposes methods to retrieve the sparse reference map, construct training or OOS query maps, materialize kernel blocks, and perform proximity-weighted prediction. The sparse matrices are built with SciPy sparse routines [50], and kernel evaluation is carried out through sparse matrix products of the form $QW^\top$. This allows the implementation to work directly with sparse leaf-coordinate representations rather than relying on a fully materialized dense proximity matrix, which is the main source of its computational efficiency.

E Compute resources. Experiments were conducted on a compute cluster using the hardware detailed in Table E.1. The system has two Intel Ice Lake CPUs with 32 cores and 64 threads. It includes 48 MB of L3 cache and supports AVX-512 and Intel Deep Learning Boost.

F Datasets. In this section we briefly describe the datasets used in our empirical experiments in Section 4. All data were kept as is without additional preprocessing. See Table F.1 for a quantitative summary of their respective sample size, number of features and class count.

[1]We stack sparse leaf-incidence maps row-wise to align with the standard row-vector convention in machine learning.

Table E.1: Compute Cluster Hardware Specifications

Component	Specification
Architecture	x86_64 (Intel Ice Lake)
CPU Model	2× Intel Xeon Gold 6326
Clock Speed	2.90 GHz / 3.50 GHz
Cores/Threads	32 Cores / 64 Threads
System Memory (RAM)	512 GB DDR4
Cache	L3: 48 MB; L2: 1.25 MB/core
Extensions	AVX-512, VNNI, Sha_NI
Virtualization	Intel VT-x with EPT

- **Airlines** is a tabular binary classification dataset of commercial flights, where the goal is to predict departure delay status from scheduled flight attributes [49].

- **Covertype** is a tabular multi-class classification dataset in which the goal is to predict the forest cover type of 30×30 meter cells from cartographic variables such as elevation, slope, aspect, hillshade, wilderness area, and soil type [3].

- **Epsilon** is a synthetic binary classification benchmark designed for the Pascal Large Scale Learning Challenge, with high-dimensional tabular features [57].

- **FashionMNIST** is a dataset of Zalando's article images consisting of a training set of 60,000 examples and a test set of 10,000 examples. Each example is a 28x28 grayscale image, associated with a label from ten types of clothes [54].

- **Higgs** is a large-scale tabular binary classification dataset of simulated particle collision events, where the goal is to distinguish signal from background using 21 low-level detector measurements and seven high-level features engineered by physicists [2].

- **PathMNIST** is a 28×28×3 color medical image dataset of colorectal cancer histology patches, derived from hematoxylin and eosin stained tissue images and organized into nine tissue classes for multi-class classification [55, 56].

- **PBMC** contains single-cell gene expression profiles from peripheral blood mononuclear cells and is widely used to study cell-type structure in high-dimensional transcriptomic data [59]. We used the preprocessed version from DensVis, where the data are already represented in principal component space [33].

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

Table F.1: Summary of the datasets used in the experiments. Columns report the dataset name, training set size, test set size (when a predefined train/test split is available), number of features, and number of classes.

Dataset	Training	Test	Features	Classes
Airlines	∼539K	–	8	2
Covertype	∼581K	–	54	7
Epsilon	400K	100K	2000	2
FashionMNIST	60K	10K	784	10
Higgs	11M	–	28	2
PathMNIST	∼97K	∼7K	2352	9
PBMC	∼69K	–	50	11
SignMNIST	∼35K	∼7K	784	24
SUSY	5M	–	18	2
TissueMNIST	∼213K	∼47K	784	8
TV News	∼130K	–	234	2

- **SignMNIST** is a grayscale 28×28 image dataset of hand gestures representing American Sign Language letters for image classification [30].
- **SUSY** is a large-scale tabular binary classification dataset of simulated particle collision events, where the goal is to distinguish supersymmetric signal from background using low-level kinematic features measured by particle detectors [2].
- **TissueMNIST** is a 28×28 grayscale medical image dataset of segmented human kidney cortex cells, derived from 3D microscopy images and organized into 8 cell categories for multi-class classification [55, 56].
- **TV News** is a multimodal dataset of broadcast news videos, where the task is to identify commercial segments using audio-visual features extracted from video shots across diverse channels and formats [52].

G Separable OOB proximities. We saw that some proximities in the existing literature are not separable (e.g., OOB and AXIL in Proposition 3.2). Since our framework shows that any proximity in the SWLC family admits a sparse factorization, it is natural to consider transforming non-separable proximities into separable ones in order to inherit near-linear scaling in the number of training samples and expose the underlying leaf-incidence maps. The challenge is that such transformations are not unique, and must preserve the underlying semantics of the original proximity. We illustrate this idea with the OOB proximity.

The standard OOB proximity P_{oob} (Appendix B.3) is not compatible with our sparse matrix factorization framework, as its normalization depends jointly on (x, x'). However, OOB remains a strong and widely used baseline, and has been shown to perform competitively in various contexts [38, 8]. It is therefore desirable to construct a separable variant that retains its core behavior while enabling scalable computation.

To this end, we introduce a separable surrogate, denoted $\tilde{P}_{\text{oob}}$, which captures the essence of the standard OOB definition while being compatible with the SWLC framework. Recall the standard OOB formulation (Appendix B.3): the context θ includes (i) per-sample, per-tree OOB indicators $o_t(\cdot) \in \{0, 1\}$, and (ii) the *shared* OOB tree counts $S(x, x') = \sum_{t=1}^{T} o_t(x)\, o_t(x')$, which record the number of trees for which *both* samples are OOB. The OOB proximity is defined as

$$P_{\text{oob}}(x, x') = \frac{1}{S(x, x')} \sum_{t=1}^{T} o_t(x)\, o_t(x')\, \mathbb{I}\big(\ell_t(x) = \ell_t(x')\big).$$

We define the following separable approximation:

$$\tilde{P}_{\text{oob}}(x, x') = \frac{T}{S(x)\, S(x')} \sum_{t=1}^{T} o_t(x)\, o_t(x')\, \mathbb{I}\big(\ell_t(x) = \ell_t(x')\big),$$

where $S(x) := \sum_{t=1}^{T} o_t(x)$. This approximation is obtained by replacing the pair-dependent normalization $S(x, x')$ with the separable proxy $S(x)S(x')/T$. Although the quantities $o_t(x)$, $S(x)$, and $P_{\text{oob}}(x, x')$ are deterministic once a specific ensemble has been realized, the following analysis interprets them as random variables induced by the ensemble generation mechanism (e.g., bootstrap resampling), with the training set held fixed. The next result shows that, under standard RF assumptions, the shared OOB count $S(x, x')$ is well approximated by its separable counterpart $S(x)S(x')/T$.

Proposition G.1 (Asymptotic separability of OOB counts). *Let $x \neq x' \in \mathcal{X}$ be two distinct training samples, and assume that trees are generated independently using standard bootstrap sampling. Define*

$$p_N = \left(1 - \frac{1}{N}\right)^N, \qquad r_N = \left(1 - \frac{2}{N}\right)^N.$$

On the event $S(x, x') > 0$, the ratio

$$\frac{\tilde{P}_{\text{oob}}(x, x')}{P_{\text{oob}}(x, x')} = \frac{S(x, x')}{S(x)S(x')/T}$$

is well defined, and satisfies

$$\frac{\tilde{P}_{\text{oob}}(x, x')}{P_{\text{oob}}(x, x')} \xrightarrow[T\to\infty]{P} \frac{r_N}{p_N^2} = 1 - \mathcal{O}\left(\frac{1}{N}\right).$$

Thus, for large T, the separable approximation differs from the exact OOB normalization by an asymptotically

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

deterministic multiplicative factor, whose bias vanishes at rate N^{-1}.

Proof. For a fixed tree t, the indicator $o_t(x)$ is Bernoulli with parameter

$$\mathbb{P}(o_t(x) = 1) = p_N = \left(1 - \frac{1}{N}\right)^N,$$

since x is OOB exactly when it is never selected among the N bootstrap draws. Likewise, for two distinct samples $x \neq x'$, $o_t(x)o_t(x')$ is Bernoulli with parameter

$$\mathbb{P}(o_t(x) = 1,\ o_t(x') = 1) = r_N = \left(1 - \frac{2}{N}\right)^N,$$

since both samples must be absent from all N draws. Because trees are generated independently, the law of large numbers gives

$$\frac{S(x)}{T} \xrightarrow{P} p_N, \qquad \frac{S(x')}{T} \xrightarrow{P} p_N, \qquad \frac{S(x,x')}{T} \xrightarrow{P} r_N.$$

Hence, on the event $S(x, x') > 0$,

$$\frac{\tilde{P}_{\text{oob}}(x,x')}{P_{\text{oob}}(x,x')} = \frac{S(x,x')/T}{(S(x)/T)(S(x')/T)} \xrightarrow{P} \frac{r_N}{p_N^2}$$

by the continuous mapping theorem, since $p_N > 0$.

It remains to simplify the limit:

$$\begin{aligned}\frac{r_N}{p_N^2} &= \frac{(1-2/N)^N}{(1-1/N)^{2N}} = \left(\frac{1-2/N}{(1-1/N)^2}\right)^N \\ &= \left(\frac{N(N-2)}{(N-1)^2}\right)^N = \left(1 - \frac{1}{(N-1)^2}\right)^N.\end{aligned}$$

Using Bernoulli's inequality,

$$\left(1 - \frac{1}{(N-1)^2}\right)^N \geq 1 - \frac{N}{(N-1)^2},$$

while trivially

$$\left(1 - \frac{1}{(N-1)^2}\right)^N \leq 1.$$

Therefore

$$0 \leq 1 - \frac{r_N}{p_N^2} \leq \frac{N}{(N-1)^2} = \mathcal{O}\left(\frac{1}{N}\right),$$

which yields

$$\frac{r_N}{p_N^2} = 1 - \mathcal{O}\left(\frac{1}{N}\right).$$

∎

Proposition G.1 shows that the approximation error is driven entirely by a mild normalization mismatch: the true shared OOB count is asymptotically separable, up to a bias of order $1/N$. In practice, for sufficiently large forests and training sets, this mismatch is negligible, making $\tilde{P}_{\text{oob}}$ a faithful and scalable surrogate of the standard OOB proximity. We validate this empirically in Section 4.1.

Remark G.2. Proposition G.1 does not apply to the case $x = x'$. Indeed, in the standard OOB definition, self-similarity is deterministic: $P_{\text{oob}}(x, x) = 1$. Therefore, to remain consistent with the original OOB proximity, we simply set the diagonal entries of $\tilde{P}_{\text{oob}}$ to 1, as done in our implementation.

H Extended runtime and memory ablation. In Fig. H.1, we provide additional scaling ablations to complement the main results in Section4.2. In addition to the ablations across proximity methods and minimum samples per leaf on Covertype, we report the same analysis on Airlines. We also include an ablation over the forest training algorithm (RF vs. ET, second row), as well as a more aggressive ablation on forest hyperparameters by restricting the tree depth d (bottom row).

Overall, the main observations remain unchanged on Airlines. However, restricting the tree depth ($d \neq$ None) significantly degrades scaling, both in time and memory, and can approach quadratic behavior in some cases (notably on Airlines). This is expected, as fixing the tree depth while increasing the number of samples leads to increasingly populated leaves. We therefore recommend tuning this parameter with care when scalability is a concern. In contrast, the minimum samples per leaf parameter appears less sensitive.

I Sanity check: accuracy of proximity-weighted classifiers. We perform a simple sanity check to verify that the kernels used in the general experiments of Section 4 yield meaningful predictions. We focus on the proximity method ablation (Fig. H.1) and report the corresponding kernel-weighted test accuracy for each method on Covertype and Airlines across varying training sizes. For reference, we also report the accuracy of the underlying forest.

Overall, kernel-weighted predictions closely match the forest accuracy. In particular, GAP almost perfectly recovers the forest performance. This is expected since GAP is designed to recover forest OOB predictions [38]. This behavior can be beneficial or detrimental in terms of accuracy: on Covertype, GAP achieves the best performance, while on Airlines it performs worse, reflecting overfitting in the underlying forest.

In contrast, while OOB tends to be noisier [38],

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

leading to lower accuracy on Covertype, it can improve performance on Airlines by mitigating overfitting. Our separable OOB thus provides a competitive and scalable alternative in practice.

J Additional forest leaf-based visualization. In Fig. J.1, we provide the same visual comparison as in Section 4.3, now on SignMNIST (A–K). The conclusions are similar: the leaf representation improves all DR pipelines. Leaf PCA gives the clearest global organization, with principal directions aligned with class semantics, whereas raw PCA remains highly noisy. Raw UMAP and raw PHATE reduce some of this noise, but tend to fragment classes into small disconnected islands and irregular trajectories. In contrast, Leaf UMAP and Leaf PHATE produce much more coherent structure. This improvement is also reflected in higher test k-NN accuracy across all leaf-based embeddings. The Leaf UMAP and Leaf PHATE embeddings remain somewhat disconnected, suggesting that larger neighborhood sizes during graph construction could further improve connectivity. More generally, our experiments indicate that Leaf PCA is a robust way to complement more complex nonlinear DR methods in leaf space by preserving a useful global view.

Acknowledgments. We would like to acknowledge the assistance of volunteers in putting together these updated SIAM Proceedings macros and example files.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

Table I.1: Test accuracy of the forest predictor and kernel-weighted predictors across training sizes on Airlines and Covertype, associated with the scaling ablation experiments in Fig. H.1.

Train size	Forest	P_{gap}	$\tilde{P}_{\text{oob}}$	P_{KeRF}	P_{original}
			Airlines		
16,384	0.629 ± 0.002	0.630 ± 0.002	0.645 ± 0.003	0.633 ± 0.002	0.645 ± 0.002
32,768	0.630 ± 0.003	0.630 ± 0.003	0.649 ± 0.003	0.633 ± 0.003	0.649 ± 0.003
65,536	0.629 ± 0.003	0.628 ± 0.003	0.651 ± 0.002	0.631 ± 0.003	0.650 ± 0.003
131,072	0.624 ± 0.001	0.624 ± 0.001	0.652 ± 0.002	0.627 ± 0.002	0.648 ± 0.002
262,144	0.618 ± 0.002	0.618 ± 0.003	0.651 ± 0.002	0.622 ± 0.002	0.646 ± 0.003
485,444	0.619 ± 0.002	0.619 ± 0.002	0.645 ± 0.002	0.625 ± 0.002	0.642 ± 0.003
			Covertype		
16,384	0.833 ± 0.002	0.833 ± 0.002	0.791 ± 0.001	0.829 ± 0.002	0.799 ± 0.001
32,768	0.863 ± 0.002	0.863 ± 0.002	0.819 ± 0.001	0.859 ± 0.001	0.826 ± 0.001
65,536	0.894 ± 0.002	0.894 ± 0.002	0.850 ± 0.003	0.890 ± 0.002	0.856 ± 0.002
131,072	0.920 ± 0.002	0.920 ± 0.002	0.881 ± 0.002	0.917 ± 0.002	0.885 ± 0.002
262,144	0.941 ± 0.002	0.941 ± 0.002	0.907 ± 0.001	0.939 ± 0.002	0.911 ± 0.001
522,910	0.957 ± 0.001	0.957 ± 0.001	0.930 ± 0.002	0.955 ± 0.001	0.932 ± 0.002

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

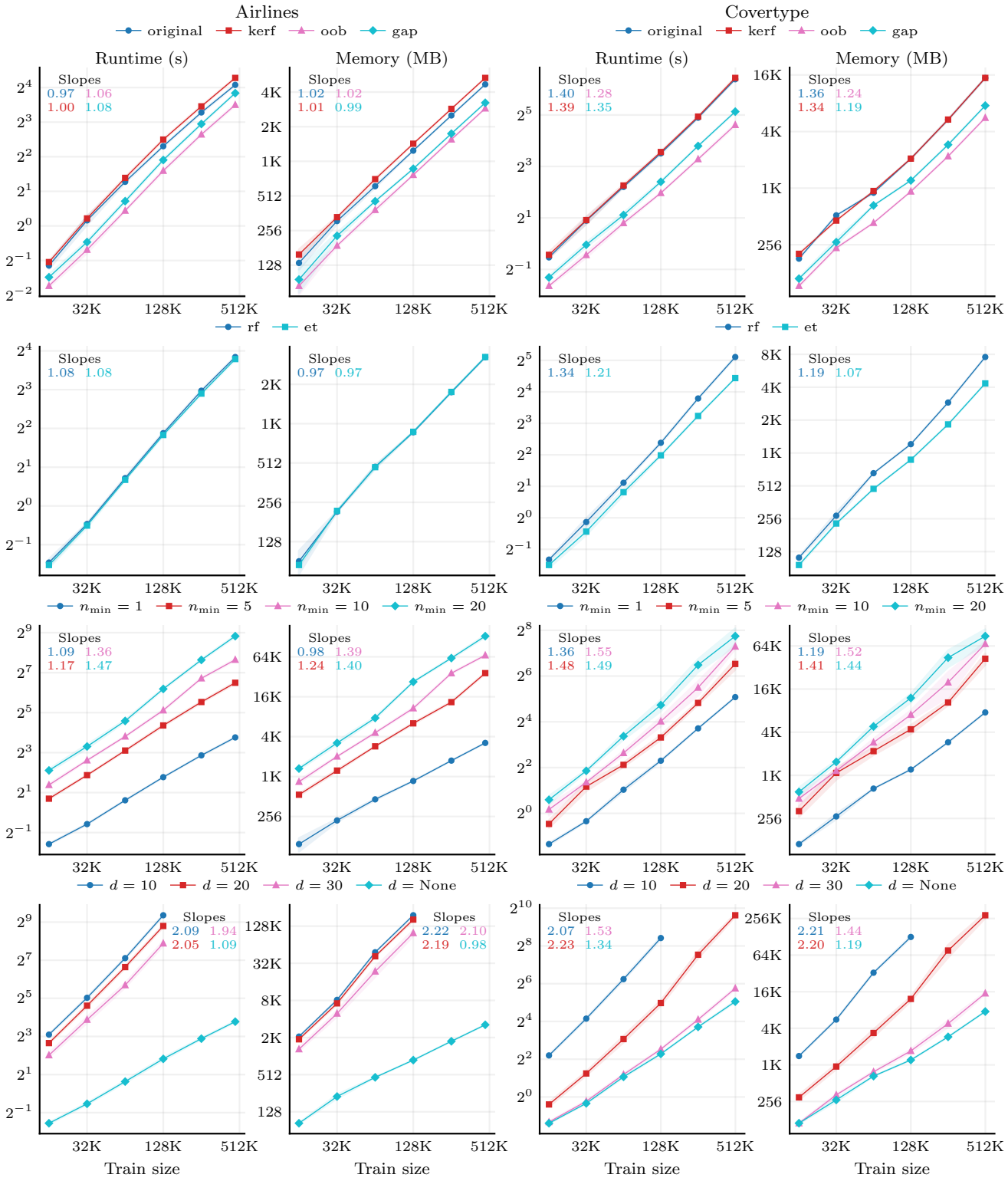


Figure H.1: Log–log runtime and memory scaling of exact kernel computation with sample size under different settings: proximity method (first row), forest type (RF and ET, second row), minimum leaf size $n_{\min}$ (third row), and maximum tree depth d (last row). Shaded bands indicate standard deviations, and slopes are estimated by linear regression. The left column shows ablations on the Airlines dataset, and the right column on Covertype. Near-linear scaling is observed in all cases, except under strong depth truncation, consistent with the fact that the complexity is driven by leaf interactions.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

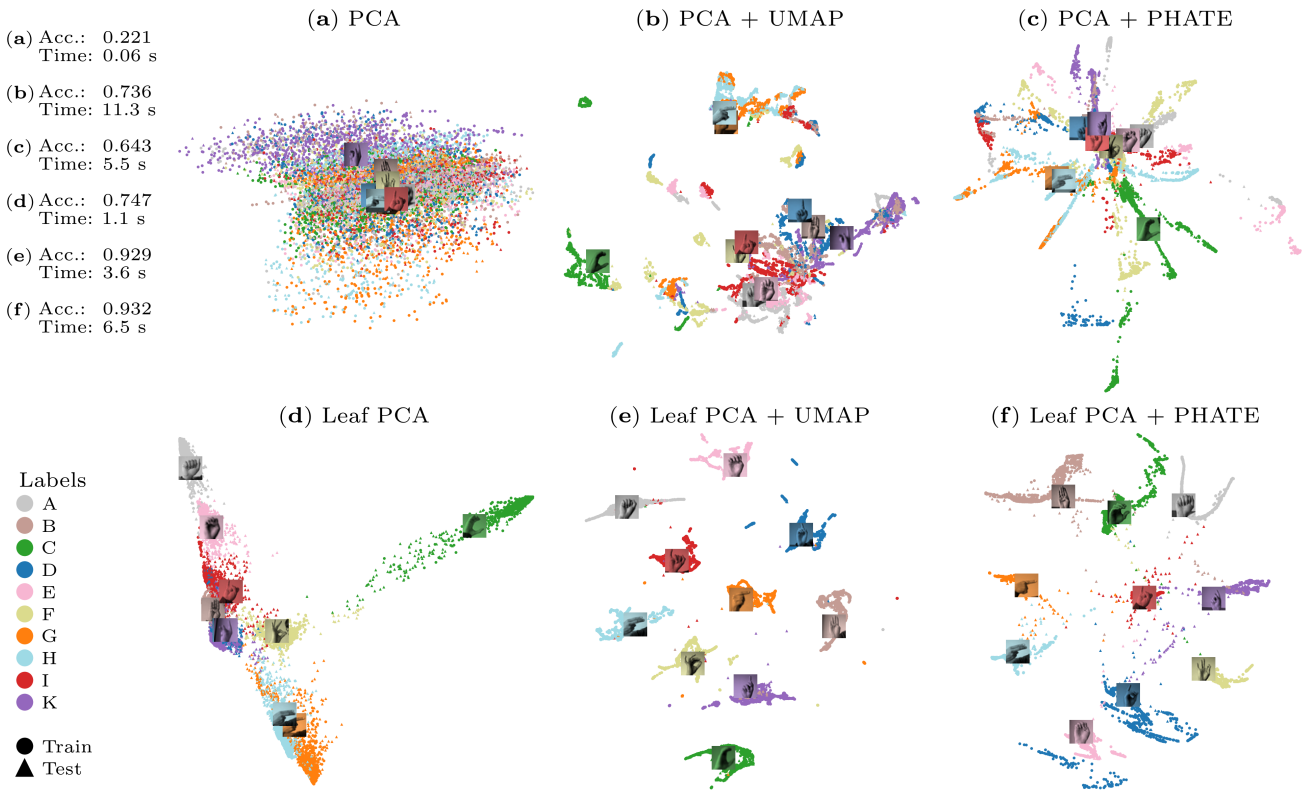


Figure J.1: Two-dimensional embeddings of the SignMNIST (A–K) train and test samples using several DR pipelines applied either to the raw pixel intensity space (top row: (a) PCA, (b) PCA+UMAP, (c) PCA+PHATE) or to the leaf-incidence space (bottom row: (d)–(f)). Colors indicate class labels, circles and triangles distinguish train and test samples, and the left legend reports the corresponding runtime and test k-NN accuracy for each panel. The classwise medoid images are overlaid at their embedding locations to summarize the dominant visual patterns of each class. All methods were trained on the full predefined SignMNIST (A–K) training split. The leaf representation improves class separability across all pipelines, as reflected by higher test k-NN accuracy. Leaf PCA gives the clearest global organization, whereas raw PCA remains noisy. Raw UMAP and raw PHATE reduce some of this noise but tend to fragment classes into small disconnected islands and irregular trajectories. In contrast, Leaf UMAP and Leaf PHATE produce more coherent class-aware structure.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

References

[1] A. Aumon, S. Ni, M. Lizotte, G. Wolf, K. R. Moon, and J. S. Rhodes, *Random forest autoencoders for guided representation learning*, in The thirty-ninth annual conference on neural information processing systems, 2025, https://openreview.net/forum?id=NjxW4m6KdH.

[2] P. Baldi, P. Sadowski, and D. Whiteson, *Searching for exotic particles in high-energy physics with deep learning*, Nature communications, 5 (2014), p. 4308.

[3] J. A. Blackard and D. J. Dean, *Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables*, Computers and Electronics in Agriculture, 24 (1999), pp. 131–151, https://doi.org/10.1016/S0168-1699(99)00046-0, https://www.sciencedirect.com/science/article/pii/S0168169999000460 (accessed 2026-04-10).

[4] L. Blanchet, R. Vitale, R. Van Vorstenbosch, G. Stavropoulos, J. Pender, D. Jonkers, F.-J. V. Schooten, and A. Smolinska, *Constructing bi-plots for random forest: Tutorial*, Analytica Chimica Acta, 1131 (2020), pp. 146–155, https://doi.org/10.1016/j.aca.2020.06.043, https://linkinghub.elsevier.com/retrieve/pii/S0003267020306917 (accessed 2026-03-24).

[5] L. Breiman, *Random forests*, Machine learning, 45 (2001), pp. 5–32.

[6] L. Breiman and A. Cutler, *Manual on setting up, using, and understanding random forests v4.0*, tech. report, University of California, Berkeley, Department of Statistics, 2003, https://www.stat.berkeley.edu/~breiman/Using_random_forests_v4.0.pdf.

[7] H. Cao, S. Bernard, R. Sabourin, and L. Heutte, *A Novel Random Forest Dissimilarity Measure for Multi-View Learning*, in 2020 25th International Conference on Pattern Recognition (ICPR), Jan. 2021, pp. 1344–1351, https://doi.org/10.1109/ICPR48806.2021.9412961, https://ieeexplore.ieee.org/document/9412961/ (accessed 2026-03-15). ISSN: 1051-4651.

[8] H. Chen, S. Farokhi, K. Bladen, H. Karimi, and K. R. Moon, *Random-forest-induced graph neural networks for tabular learning*, 2026, https://arxiv.org/abs/2602.24224. arXiv: 2602.24224 [cs.GR].

[9] A. Davies and Z. Ghahramani, *The Random Forest Kernel and other kernels for big data from random partitions*, Feb. 2014, https://doi.org/10.48550/arXiv.1402.4293, http://arxiv.org/abs/1402.4293 (accessed 2026-03-15). arXiv:1402.4293 [stat].

[10] L. Duvaux, Q. Geissmann, K. Gharbi, J.-J. Zhou, J. Ferrari, C. M. Smadja, and R. K. Butlin, *Dynamics of Copy Number Variation in Host Races of the Pea Aphid*, Molecular Biology and Evolution, 32 (2015), pp. 63–80, https://doi.org/10.1093/molbev/msu266, https://academic.oup.com/mbe/article-lookup/doi/10.1093/molbev/msu266 (accessed 2026-01-04).

[11] C. Englund and A. Verikas, *A novel approach to estimate proximity in a random forest: An exploratory study*, Expert Systems with Applications, 39 (2012), pp. 13046–13050, https://doi.org/10.1016/j.eswa.2012.05.094, https://linkinghub.elsevier.com/retrieve/pii/S095741741200810X (accessed 2026-03-15).

[12] S. Farokhi, H. Chen, K. Moon, and H. Karimi, *Advancing Tabular Data Classification with Graph Neural Networks: A Random Forest Proximity Method*, in 2024 IEEE International Conference on Big Data (BigData), Dec. 2024, pp. 7011–7020, https://doi.org/10.1109/BigData62323.2024.10825972, https://ieeexplore.ieee.org/document/10825972 (accessed 2026-01-04).

[13] J. H. Friedman, *Greedy Function Approximation: A Gradient Boosting Machine*, The Annals of Statistics, 29 (2001), pp. 1189–1232, https://www.jstor.org/stable/2699986 (accessed 2026-01-04).

[14] Y. Ganjisaffar, R. Caruana, and C. V. Lopes, *Bagging gradient-boosted trees for high precision, low variance ranking models*, in Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval, SIGIR '11, New York, NY, USA, July 2011, Association for Computing Machinery, pp. 85–94, https://doi.org/10.1145/2009916.2009932, https://dl.acm.org/doi/10.1145/2009916.2009932 (accessed 2026-03-17).

[15] P. Geertsema and H. Lu, *Instance-based Explanations for Gradient Boosting Machine Predictions with AXIL Weights*, Jan. 2023, https://doi.org/10.48550/arXiv.2301.01864, http:

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

//arxiv.org/abs/2301.01864 (accessed 2026-03-14). arXiv:2301.01864 [cs].

[16] P. Geurts, D. Ernst, and L. Wehenkel, *Extremely randomized trees*, Machine Learning, 63 (2006), pp. 3–42, https://doi.org/10.1007/s10994-006-6226-1, https://doi.org/10.1007/s10994-006-6226-1 (accessed 2026-01-04).

[17] N. Halko, P.-G. Martinsson, and J. A. Tropp, *Finding structure with randomness: Stochastic algorithms for constructing approximate matrix decompositions.(2009)*, URL http://arxiv.org/abs/0909.4061. oai: arXiv. org, 909 (2009).

[18] K. Hassine, A. Erbad, and R. Hamila, *Important Complexity Reduction of Random Forest in Multi-Classification Problem*, in 2019 15th International Wireless Communications & Mobile Computing Conference (IWCMC), June 2019, pp. 226–231, https://doi.org/10.1109/IWCMC.2019.8766544, https://ieeexplore.ieee.org/document/8766544 (accessed 2026-01-04).

[19] T. Hastie, R. Tibshirani, and J. Friedman, *Random forests*, in The elements of statistical learning: Data mining, inference, and prediction, Springer New York, New York, NY, 2009, pp. 587–604, https://doi.org/10.1007/978-0-387-84858-7_15, https://doi.org/10.1007/978-0-387-84858-7_15.

[20] H. Hazimeh, N. Ponomareva, P. Mol, Z. Tan, and R. Mazumder, *The Tree Ensemble Layer: Differentiability meets Conditional Computation*, in Proceedings of the 37th International Conference on Machine Learning, PMLR, Nov. 2020, pp. 4138–4148, https://proceedings.mlr.press/v119/hazimeh20a.html (accessed 2026-04-03).

[21] M. He, F. He, L. Shi, X. Huang, and J. A. K. Suykens, *Learning With Asymmetric Kernels: Least Squares and Feature Interpretation*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 45 (2023), pp. 10044–10054, https://doi.org/10.1109/TPAMI.2023.3257351, https://ieeexplore.ieee.org/document/10070836 (accessed 2026-03-22).

[22] R. A. Horn and C. R. Johnson, *Matrix analysis*, Cambridge university press, 2012.

[23] J. Jeyapaulraj, D. Desai, D. Mehta, P. Chu, S. Pasquali, and P. Sommer, *Supervised similarity learning for corporate bonds using Random Forest proximities*, in Proceedings of the Third ACM International Conference on AI in Finance, ICAIF '22, New York, NY, USA, Oct. 2022, Association for Computing Machinery, pp. 411–419, https://doi.org/10.1145/3533271.3561736, https://dl.acm.org/doi/10.1145/3533271.3561736 (accessed 2026-03-20).

[24] C. Kuchar, *RFX: High-Performance Random Forests with GPU Acceleration and QLORA Compression*, Nov. 2025, https://doi.org/10.48550/arXiv.2511.19493, http://arxiv.org/abs/2511.19493 (accessed 2026-03-15). arXiv:2511.19493 [cs].

[25] J. Lee, *Patient-Specific Predictive Modeling Using Random Forests: An Observational Study for the Critically Ill*, JMIR Medical Informatics, 5 (2017), p. e6690, https://doi.org/10.2196/medinform.6690, https://medinform.jmir.org/2017/1/e3 (accessed 2026-01-04). Company: JMIR Medical Informatics Distributor: JMIR Medical Informatics Institution: JMIR Medical Informatics Label: JMIR Medical Informatics.

[26] R. B. Lehoucq, D. C. Sorensen, and C. Yang, *ARPACK users' guide: solution of large-scale eigenvalue problems with implicitly restarted Arnoldi methods*, SIAM, 1998.

[27] A. Liaw, M. Wiener, and others, *Classification and regression by randomForest*, R news, 2 (2002), pp. 18–22.

[28] G. Louppe, *Understanding Random Forests: From Theory to Practice*, June 2015, https://doi.org/10.48550/arXiv.1407.7502, http://arxiv.org/abs/1407.7502 (accessed 2026-01-04). arXiv:1407.7502 [stat].

[29] S. M. Lundberg and S.-I. Lee, *A Unified Approach to Interpreting Model Predictions*, in Advances in Neural Information Processing Systems, vol. 30, Curran Associates, Inc., 2017, https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html (accessed 2026-03-23).

[30] D. Massey, *Sign language MNIST*, 2017, https://www.kaggle.com/datasets/datamunge/sign-language-mnist.

[31] L. McInnes, J. Healy, N. Saul, and L. Grossberger, *UMAP: Uniform manifold approximation and projection*, Journal of Open

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

Source Software, 3 (2018), p. 861, https://doi.org/10.21105/joss.00861, https://doi.org/10.21105/joss.00861.

[32] K. R. Moon, D. Van Dijk, Z. Wang, S. Gigante, D. B. Burkhardt, W. S. Chen, K. Yim, A. v. d. Elzen, M. J. Hirn, R. R. Coifman, and others, *Visualizing structure and transitions in high-dimensional biological data*, Nature biotechnology, 37 (2019), pp. 1482–1492.

[33] A. Narayan, B. Berger, and H. Cho, *Density-preserving data visualization unveils dynamic patterns of single-cell transcriptomic variability*, bioRxiv : the preprint server for biology, (2020), pp. 2020–05.

[34] M. A. Olson and A. J. Wyner, *Making Sense of Random Forest Probabilities: a Kernel Perspective*, Dec. 2018, https://doi.org/10.48550/arXiv.1812.05792, http://arxiv.org/abs/1812.05792 (accessed 2026-03-15). arXiv:1812.05792 [stat].

[35] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, *Scikit-learn: Machine learning in Python*, Journal of Machine Learning Research, 12 (2011), pp. 2825–2830.

[36] J. S. Rhodes, *Supervised Manifold Learning via Random Forest Geometry-Preserving Proximities**, in 2023 International Conference on Sampling Theory and Applications (SampTA), July 2023, pp. 1–5, https://doi.org/10.1109/SampTA59647.2023.10301399, https://ieeexplore.ieee.org/document/10301399/ (accessed 2026-04-04). ISSN: 2694-0108.

[37] J. S. Rhodes, A. Aumon, S. Morin, M. Girard, C. Larochelle, B. Lahav, E. Brunet-Ratnasingham, W. Zhang, A. Cutler, A. Zhou, D. E. Kaufmann, S. Zandee, A. Prat, G. Wolf, and K. R. Moon, *Gaining Biological Insights through Supervised Data Visualization*, Nov. 2023, https://doi.org/10.1101/2023.11.22.568384, https://www.biorxiv.org/content/10.1101/2023.11.22.568384v1 (accessed 2026-01-04). Pages: 2023.11.22.568384 Section: New Results.

[38] J. S. Rhodes, A. Cutler, and K. R. Moon, *Geometry- and Accuracy-Preserving Random Forest Proximities*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 45 (2023), pp. 10947–10959, https://doi.org/10.1109/TPAMI.2023.3263774, https://ieeexplore.ieee.org/document/10089875 (accessed 2026-01-04).

[39] J. S. Rhodes and A. G. Rustad, *Random Forest-Supervised Manifold Alignment*, IEEE Computer Society, Dec. 2024, pp. 3309–3312, https://doi.org/10.1109/BigData62323.2024.10825663, https://www.computer.org/csdl/proceedings-article/bigdata/2024/10825663/23ykS7N2xR6 (accessed 2026-01-04).

[40] M. T. Ribeiro, S. Singh, and C. Guestrin, *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*, in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16, New York, NY, USA, Aug. 2016, Association for Computing Machinery, pp. 1135–1144, https://doi.org/10.1145/2939672.2939778, https://dl.acm.org/doi/10.1145/2939672.2939778 (accessed 2026-03-23).

[41] J. Rosaler, D. Desai, B. Sarmah, D. Vamvourellis, D. Onay, S. Pasquali, and D. Mehta, *Enhanced Local Explainability and Trust Scores with Random Forest Proximities*, in Proceedings of the 5th ACM International Conference on AI in Finance, ICAIF '24, New York, NY, USA, Nov. 2024, Association for Computing Machinery, pp. 521–529, https://doi.org/10.1145/3677052.3698615, https://dl.acm.org/doi/10.1145/3677052.3698615 (accessed 2026-03-23).

[42] E. Scornet, *Random forests and kernel methods*, IEEE Transactions on Information Theory, 62 (2016), pp. 1485–1500.

[43] B. Shaw, J. Rhodes, S. F. Boubrahimi, and K. R. Moon, *Forest Proximities for Time Series*, in Intelligent Systems and Applications, K. Arai, ed., Cham, 2026, Springer Nature Switzerland, pp. 588–598, https://doi.org/10.1007/978-3-032-07109-5_40.

[44] S. Sonnenburg, G. Rätsch, C. Schäfer, and B. Schölkopf, *Large scale multiple kernel learning*, The Journal of Machine Learning Research, 7 (2006), pp. 1531–1565.

[45] J. A. Suykens, *SVD revisited: A new variational principle, compatible feature maps and nonlinear extensions*, Applied and Computational Harmonic Analysis, 40 (2016), pp. 600–609,

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

https://doi.org/10.1016/j.acha.2015.09.004, https://linkinghub.elsevier.com/retrieve/pii/S1063520315001360 (accessed 2026-03-23).

[46] S. Tan, M. Soloviev, G. Hooker, and M. T. Wells, *Tree Space Prototypes: Another Look at Making Tree Ensembles Interpretable*, in Proceedings of the 2020 ACM-IMS on Foundations of Data Science Conference, FODS '20, New York, NY, USA, Oct. 2020, Association for Computing Machinery, pp. 23–34, https://doi.org/10.1145/3412815.3416893, https://doi.org/10.1145/3412815.3416893 (accessed 2026-01-04).

[47] Q. Tao, F. Tonin, A. Lambert, Y. Chen, P. Patrinos, and J. Suykens, *Learning in Feature Spaces via Coupled Covariances: Asymmetric Kernel SVD and Nyström method*, in Proceedings of the 41st International Conference on Machine Learning, PMLR, July 2024, pp. 47929–47947, https://proceedings.mlr.press/v235/tao24b.html (accessed 2026-03-23).

[48] R. C. Team, *RA language and environment for statistical computing, R Foundation for Statistical*, Computing. Archives for Scientific Computing, (2020).

[49] J. Vanschoren, J. N. Van Rijn, B. Bischl, and L. Torgo, *OpenML: networked science in machine learning*, ACM SIGKDD Explorations Newsletter, 15 (2014), pp. 49–60.

[50] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, I. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, *SciPy 1.0: Fundamental algorithms for scientific computing in python*, Nature Methods, 17 (2020), pp. 261–272, https://doi.org/10.1038/s41592-019-0686-2.

[51] B. D. Vu, J. Kapar, M. N. Wright, and D. Watson, *Autoencoding random forests*, in The thirty-ninth annual conference on neural information processing systems, 2025, https://openreview.net/forum?id=D7zMB5uEHw.

[52] A. Vyas, R. Kannao, V. Bhargava, and P. Guha, *Commercial Block Detection in Broadcast News Videos*, in Proceedings of the 2014 Indian Conference on Computer Vision Graphics and Image Processing, ICVGIP '14, New York, NY, USA, Dec. 2014, Association for Computing Machinery, pp. 1–7, https://doi.org/10.1145/2683483.2683546, https://dl.acm.org/doi/10.1145/2683483.2683546 (accessed 2026-04-09).

[53] L. S. Whitmore, A. George, and C. M. Hudson, *Explicating feature contribution using Random Forest proximity distances*, July 2018, https://doi.org/10.48550/arXiv.1807.06572, http://arxiv.org/abs/1807.06572 (accessed 2026-03-18). arXiv:1807.06572 [stat].

[54] H. Xiao, K. Rasul, and R. Vollgraf, *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*, Sept. 2017, https://doi.org/10.48550/arXiv.1708.07747, http://arxiv.org/abs/1708.07747 (accessed 2026-01-05). arXiv:1708.07747 [cs].

[55] J. Yang, R. Shi, and B. Ni, *Medmnist classification decathlon: A lightweight automl benchmark for medical image analysis*, in 2021 IEEE 18th international symposium on biomedical imaging (ISBI), IEEE, 2021, pp. 191–195.

[56] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, *Medmnist v2-a large-scale lightweight benchmark for 2d and 3d biomedical image classification*, Scientific data, 10 (2023), p. 41.

[57] G.-X. Yuan, C.-H. Ho, and C.-J. Lin, *An improved glmnet for l1-regularized logistic regression*, in Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining, 2011, pp. 33–41.

[58] P. Zhao, X. Su, T. Ge, and J. Fan, *Propensity score and proximity matching using random forest*, Contemporary Clinical Trials, 47 (2016), pp. 85–92, https://doi.org/10.1016/j.cct.2015.12.012, https://linkinghub.elsevier.com/retrieve/pii/S1551714415301439 (accessed 2026-03-24).

[59] G. X. Zheng, J. M. Terry, P. Belgrader, P. Ryvkin, Z. W. Bent, R. Wilson, S. B. Ziraldo, T. D. Wheeler, G. P. McDermott, J. Zhu, and others, *Massively parallel digital transcriptional profiling of single cells*, Nature communications, 8 (2017), p. 14049.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited